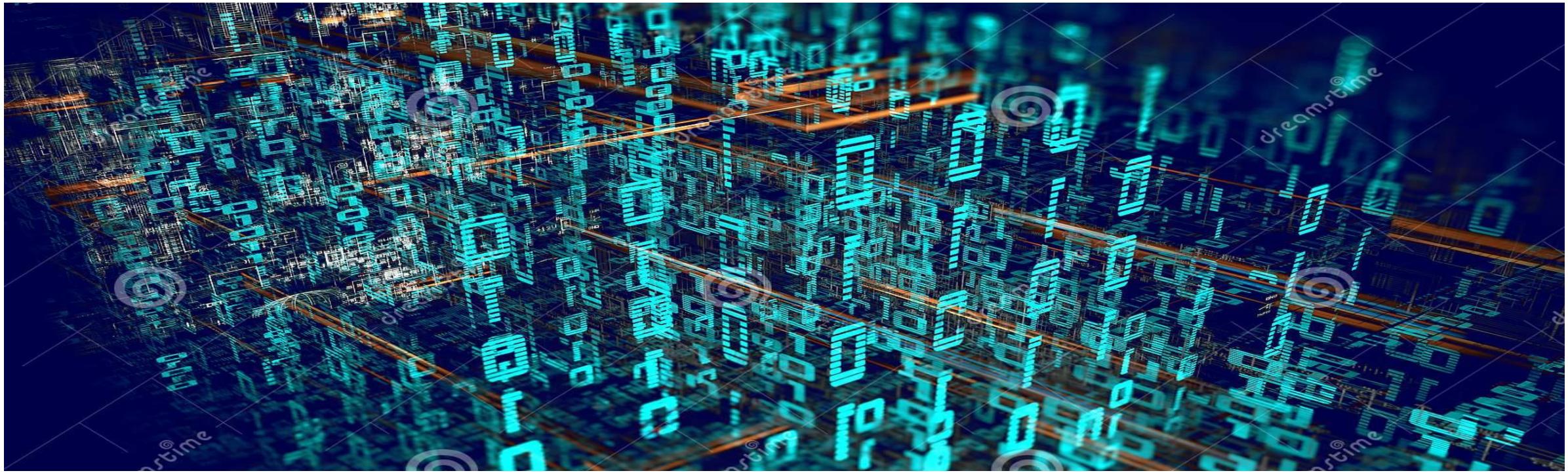
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



MODULE 4

INTERMEDIATE CODE GENERATION

Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

Types – Declarations

- Applications of types are grouped under
  - ***checking***
  - ***Translation***

Type checking

- It **uses logical rules** to reason about the behavior of a program at run time.
- It ensures that the operands match the type that an operation expects.

- Example:
  The **&& operator** in Java expects two booleans as operands and returns a boolean result.

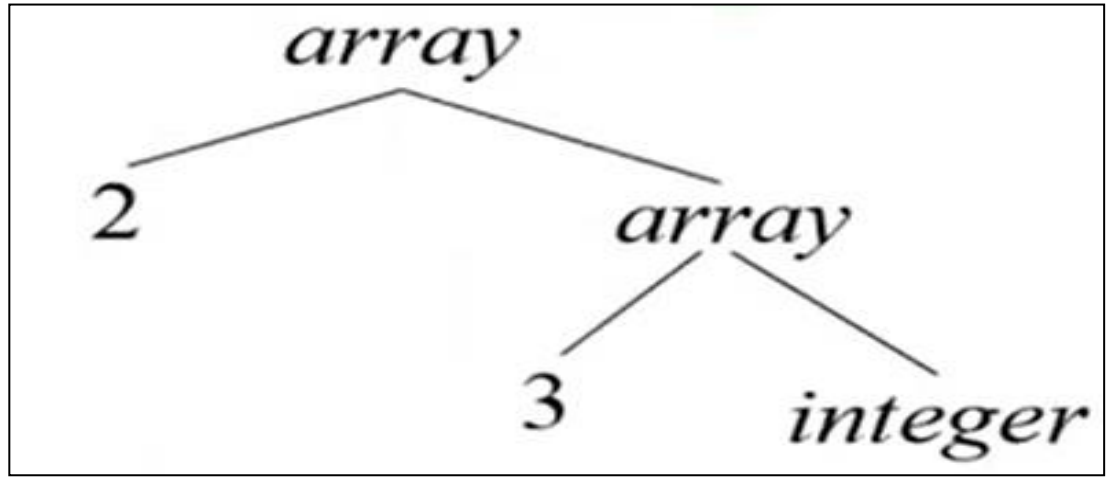
```
A = -10, B=20;

C= (A  && B ); // true && true

C = 1 //true
```

Translation Applications

- From the ***type of a name***, a compiler can ***determine the storage*** that will be needed for that name at run time.
- **Type information** is also needed to calculate the address denoted by an ***array reference***, to ***insert explicit type conversions***, and to choose the ***right version of an arithmetic operator***, among other things.

1. Type Expressions

- Types have ***structure***, which is represented using type expressions.
- A type expression is either ***a primary(basic) type*** or is formed by applying a ***type constructor operator*** to a type expression.
- The basic types and constructors depend on the language to be checked.
- Example: **Type Array**
  - An array of type **int[2][3]** that is read as ***array of 2 arrays of 3 integers*** *each*.
  - It is written as a ***type expression*** as **array(2, array(3, integer))**
  - The operator array always takes two parameters, a ***number*** and a ***type***



The definitions of type expressions

- A **basic type** expression : Include ***boolean***, ***char***, ***integer***, ***float*** and ***void***.

E→literal	{E.type = char }
E→ num	{E.type = integer}
E→ id	{E.type = lookup(id.entry) }

- A **type name** is a type expression (typedef)

E→ a {E.name =a}

- A type ***expression*** can be formed by applying the ***array type constructor*** to a number and a type expression.

If **E** is a **type expression** and **n** is an **int**, then

array(n, E)

is a type expression indicating array with E and indices in the range 0 … n - 1.

int a[2][3] is array of 2 arrays of 3 integers

Functional style : **array(2, array(3, integer)**

Type Constructors

Arrays

Records

Sets

Pointers

Functions

- A ***record*** is a data structure that holds multiple named fields.
  - A type expression can be formed by applying the ***record type constructor*** to the field names and their types.
  - Record types can be implemented by applying the constructor record to a symbol table containing entries for the fields.

If **abc** and **xyz** are two **type names**, if **E1** and **E2** are **type expressions** then

record(abc: E1, xyz: E2)

is a type expression indicating the type of an element of the Cartesian product of T1 and T2.

Example :

Suppose we define a ***record*** with two fields:

- name of type string.
- age of type integer.

***record*(name: string, age: integer)**

The record can store different instances like:

{name: "Alice", age: 30}
{name: "Bob", age: 25}

In a C-like syntax, it can be represented as a struct

```
struct Person
{
    string name;
    int age;
};
```

- A type ***expression*** can be formed by using the type constructor "→" for ***function types***.
- If **E1** and **E2** are type expressions, it means there is a function that takes an argument of type E1 and returns a result of type E2.

  E1: Represents the **input type** of the function

  E2: Represents the **output type** of the function.

Example:

Function from Integer to String:

- If E1 is int and E2 is string, then, Type Expression: **int → string**
  - This indicates a function that takes an integer and returns a string.

Function from String to Boolean:

- If E1 is string and E2 is bool, then: Type Expression: **string → bool**
  - This indicates a function that takes a string and returns a boolean value.

{ E→type ={ if E2.type = T1 and E1.type = T1→T2 then T2 else type_errror }}

- E1 and E2 are sub-expressions.
- T1 and T2 represent types.
- If both sub-expressions (E1 and E2) are of type T1, then the resulting type of the expression is T2. If not, a type error occurs.

{E → type = {if E2.type = int and E1.type = int → int then int else type_error}}

- If ***s*** and ***t*** are **type expressions**, their ***Cartesian product s x t*** is a type expression.
- we can create a new type that consists of pairs formed from the elements of the types s and t.

Example

- Let s = int and t = float
- ***The Cartesian product s x t*** *is represented as*

int x flaot

- This denotes a type expression representing a pair consisting of an integer and a float.
- When **T** is a **type expression**, **pointer(T)** is a type expression that denotes "***pointer to an object of type T***"

E→*E1

```
{E.type = {if E.type = ptr(T) then
                    T
          else
                    type_error } }
```

2. Type Equivalence

- Type-checking rules are of the form; *'if two type expressions are equal then return a certain type, else return an error'*.
- When **graphs** represent type expressions, two types are structurally equivalent if and only if one of the following conditions is true:
  - ***They are of the same basic type.***
  - ***They are formed by applying the identical constructor to structurally equivalent types.***
  - ***One is a type name that refers to the other.***

Name equivalence
char a,b;
a='A'
b=a

Structural equivalence
struct a,b;

int a[2][3] is not equivalent to **int b[3][2]**
int a is not equivalent to **char b[4]**
struct {int, char} is not equivalent to **struct{char,int }**
int* is not equivalent to ***void****

3. Declarations

- Types and declarations use simplified grammar to declare a single name at a time.
- A simple ***type declaration grammar***

$$D \rightarrow T\ L$$
$$T \rightarrow int \mid float$$
$$L \rightarrow L_1,\ id \mid id$$

- ***Syntax Directed Definition*** for simple type declaration

Production	Semantic Rule
1. $D \rightarrow T\ L$	$L.inh = T.type$
2. $T \rightarrow int$	$T.type = integer$
3. $T \rightarrow float$	$T.type = float$
4. $L \rightarrow L_1,\ id$	$L_1.inh = L.inh$ $addType(id.entry,\ L.inh)$
5. $L \rightarrow id$	$addType(id.entry,\ L.inh)$

- Consider the grammar containing **declaration with list of names**

$$D \rightarrow T\ id;\ D \mid \epsilon$$
$$T \rightarrow B\ C \mid record\ '\{'\ D\ '\}'$$
$$B \rightarrow int \mid float$$
$$C \rightarrow \epsilon \mid [num]\ C$$

$$D \rightarrow T\ L$$
$$T \rightarrow int \mid float$$
$$L \rightarrow L_1,\ id \mid id$$

- The fragment of the this grammar deals with ***basic and array types*** which was used to illustrate inherited attributes.
- Here we consider **storage layout** as well as **types**
- Nonterminal ***D*** generates a **sequence of declarations**
- Nonterminal ***T*** generates **basic, array, or record types**.
- Nonterminal ***B*** generates **one of the basic types int and float**.
- Nonterminal **C** generates **strings of zero or more integers**, each integer surrounded by brackets.
- An **array type** consists of a basic type specified by *B*, followed by array components specified by nonterminal *C*.
- A **record type** is a sequence of declarations for fields of the record surrounded by curly braces.

- **Example**

$D \rightarrow T\ id;\ D \mid \epsilon$

$T \rightarrow B\ C \mid record\ `\{`\ D\ `\}`$

$B \rightarrow int \mid float$

$C \rightarrow \epsilon \mid [num]\ C$

$D \rightarrow T\ L$

$T \rightarrow int \mid float$

$L \rightarrow L_1,\ id \mid id$

```
int x;
float y;
    record {
            int z;
           float a[10];
    record {
           int b;
    }
}
```

4. Storage Layout for Local Names

- Given a type name, we can determine the amount of storage that is needed for the name at runtime.
- During compile time we use these amounts to assign each name a relative address.
- ***Type*** and ***relative addresses*** are stored in a **symbol table entry** for the name.
- ***Varying-length data*** such as strings or data whose size cannot be determined until runtime such as dynamic arrays is handled by keeping a **fixed amount of storage** for a **pointer to data**.
- Assuming that storage is in blocks of contiguous bytes whereby a byte is the smallest unit of addressable memory.
- **Multibyte objects** are stored in consecutive bytes and given the address of the first byte.
- The **width of a type** is the number of storage units needed for objects of that type.
- For easy access, storage for aggregates such as arrays and classes is allocated in one contiguous block of bytes

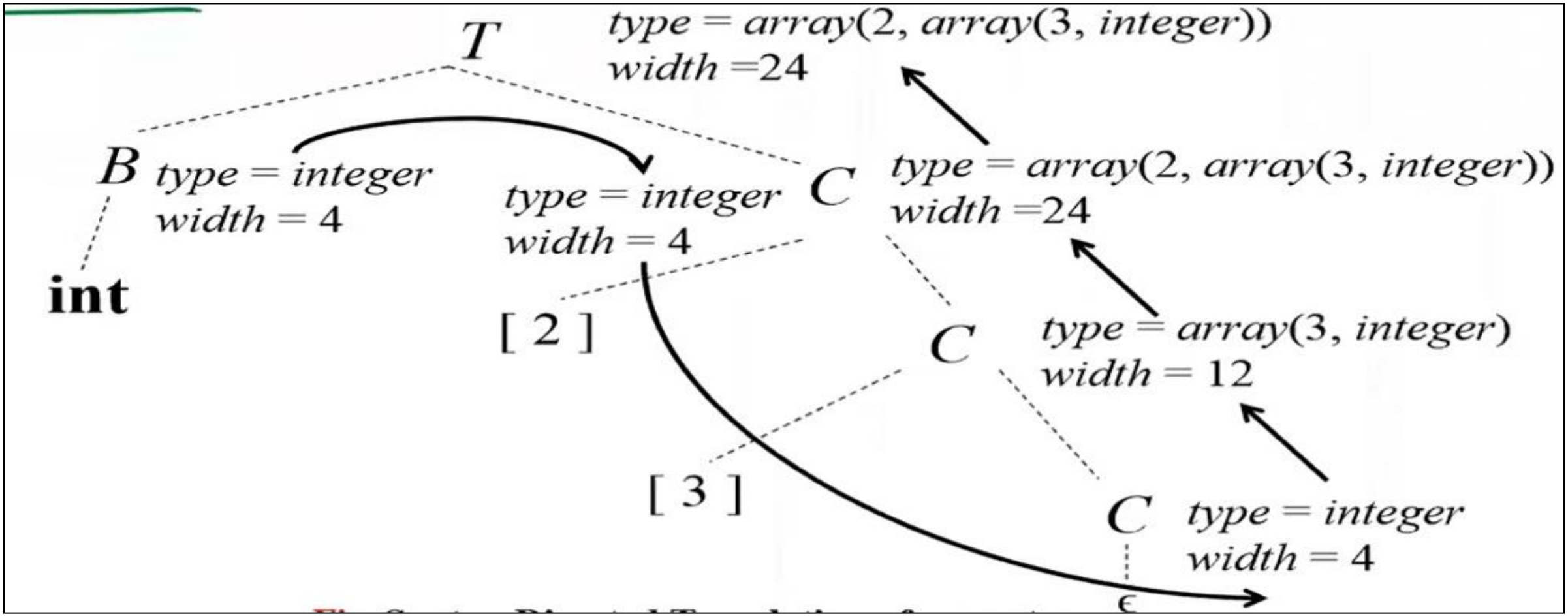
- The **translation scheme** (SDT) shown below computes ***types*** and their ***widths*** for **basic and array types**

Production	Action
$T \rightarrow B$	{ t = B.type; w = B.width; }
$T \rightarrow C$	{T.type = C.type; T. width = C.width; }
$B \rightarrow$ int	{ B.type = integer; B.width = 4; }
$B \rightarrow$ float	{ B.type = float; B.width = 8; }
$C \rightarrow \epsilon$	{ C.type = t; C.width = w; }
$C \rightarrow$ [num] C_1	{ C.type = array(num.value; C_1.type); C.width = num.value x C_1.width; }

- *SDT* uses ***synthesized attributes*** ***type*** and ***width*** for each non-terminal and ***two variables*** ***t*** and ***w*** to pass type and width information from *B* node in a parse tree to a node for the production $C \rightarrow \varepsilon$
- *t* and *w* would be inherited attributes for *C*.

- The body of the ***T-production*** consists of a **non-terminal *B***, an **action**, and a ***non-terminal C*** that appears on the next line.
  - The action between *B* and *C* sets *t* to *B-type* and *w* to *B.width*.
- If B → int, B.type is set to an integer and B.width is set to 4 which is the width of an integer.
- If B → float, B.type is float and B.width is 8, which is the width of a float.
- Productions for *C* determine whether *T* generates a basic type or an array type.
  - If C →ε, then t is C.type and w is C.width.
  - *If C* specifies an array component then he action for *C → [num] C1* forms *C.type* by applying the type constructor array to the operands *num.value* and *C1.type*.
- The width of an array is obtained by multiplying the width of an element by the number of elements in the array.

- The parse tree for the type **int [2] [3]** for SDT of array type is given below



5. Sequences of Declarations (Self Study Topic)

6. Fields in Records and Classes (Self Study Topic)